IoT向けHW/SW同時最適化ブロック暗号の設計

田中 篤[1,a)]　五十部 孝典[1]

概要：Massive IoTでは，センサ情報等の小さなペイロードを低コストで保護する必要がある．小ペイロードでは鍵スケジュールを複数ブロックに分散できず，RAMも限られるため，オンザフライ鍵スケジュールを含む1ブロック暗号化の効率が重要となる．暗号処理は専用ハードウェア（HW）と組込みプロセッサ（SW）の双方で行われるため，両環境を設計段階から最適化する必要がある．本稿では，両環境の演算特性を相補的に活用し，HW/SWを同時最適化した128ビットブロック暗号を提案する．Cortex-M3/M4向け実装では，鍵更新を含む暗号化処理を13本の汎用レジスタ内で完結させ，入出力を除くデータメモリアクセスをゼロにする．SWでは，1ラウンドループ実装でありながら，Speck-128-128のアンロール実装と同等の暗号化性能を約1/10のコードサイズで達成する．15 nm ASIC評価でもMidori-128およびQARMAv2-128と競争力のある性能を示し，AES-128に対してはHW/SW双方で実装コストを大幅に削減する．以上より，軽量暗号を単一プリミティブで実現し，HW/SW同時最適化の有効性を示す．

キーワード：Massive IoT，軽量暗号，Cortex-M3/M4

Design of a Hardware–Software Co-Optimized Block Cipher for IoT

Atsushi Tanaka[1,a)]　Takanori Isobe[1]

Abstract: Massive IoT requires the protection of small payloads, such as sensor data, at low cost. For such short payloads, the cost of the key schedule cannot be amortized over multiple blocks, while available RAM is also limited. Therefore, the efficiency of single-block encryption, including an on-the-fly key schedule, is particularly important. Since cryptographic operations are performed in both dedicated hardware (HW) and software (SW) on embedded processors, both implementation environments should be optimized from the design stage. In this paper, we propose a 128-bit block cipher that jointly optimizes HW and SW by complementarily exploiting the computational characteristics of both environments. On Cortex-M3/M4, the entire encryption process, including key updates, is completed within 13 general-purpose registers, eliminating all data-memory accesses except for input and output. Our single-round-loop SW implementation achieves encryption performance comparable to an unrolled implementation of Speck128. In a 15 nm ASIC evaluation, the proposed cipher also achieves competitive performance with Midori-128 and QARMAv2-128, while substantially reducing implementation costs compared with AES-128 in both HW and SW. These results demonstrate that a single primitive can achieve performance competitive with high-performance ciphers specialized for each environment, validating the effectiveness of joint HW/SW optimization.

1. はじめに

1.1 背景

IoT（Internet of Things）の普及に伴い，センサ，メータ，トラッカなど多数のエッジデバイスがネットワークに接続され，それらが生成する情報の保護が重要となっている．特にMassive IoTでは，演算性能，メモリ容量，消費電力に厳しい制約を持つデバイス上で暗号処理を実行する必要があり，これまでに多数の軽量ブロック暗号が提案されてきた．

IoTデバイスの暗号処理は，専用ハードウェアと組み込みプロセッサ上のソフトウェアの双方で実行される．例えば，LoRaWAN [1] のリファレンス実装LoRaMAC-nodeでは，

[1] 大阪大学
The University of Osaka
[a)] a.tanaka@ist.osaka-u.ac.jp

© 2026 Information Processing Society of Japan

This paper is work in progress and not peer-reviewed.

Secure Element 等のハードウェアを利用する構成とソフトウェア実装の双方が考慮されている．また，STM32WL シリーズのように，Cortex-M 系プロセッサと AES ハードウェアアクセラレータを同一 SoC 上に備えるデバイスも存在する．このため，IoT 向け暗号では，ハードウェア（HW）とソフトウェア（SW）の双方を考慮した設計が求められる．

1.2 既存研究の課題

HW と SW では，効率的に実行できる演算の性質が異なる．例えば，固定 bit permutation は HW では主に配線置換として低コストに実現できる一方，SW では複数のマスクやシフト等を必要とする場合がある．一方，XOR，シフト，ローテーションなどのワード演算は汎用プロセッサ上で効率的に実行できる．このため，一方の実装環境に最適化された暗号構造が，他方でも同様に高い実装効率を示すとは限らない．

既存の軽量ブロック暗号では，HW 向けの低エネルギー実装を重視した Midori [2] や，低遅延実装を重視した QARMAv2 [3]，SW 向けにワード演算を中心として高速化された Speck [4] などが提案されている．一方で，HW/SW 双方の演算コストを設計段階から同時に考慮し，両環境で低遅延かつ低エネルギーな実装を追求する設計は十分に検討されていない．

さらに，Massive IoT では暗号化対象となるペイロードが短い．例えば Sigfox [5] ではペイロードは最大 12 byte であり，CTR モードでは 1 つの 128 bit ブロックによるキーストリーム生成で処理できる．このような環境では，多数ブロックのスループットよりも，鍵処理を含む 1 ブロックのレイテンシが重要となる．したがって，ラウンド鍵を事前展開せず，暗号化と並行して生成する on-the-fly 鍵スケジュールを含めた 1 ブロック処理全体の効率を考慮する必要がある．

また，組み込み SW では，アンロールによる高速化とコードサイズの増加との間にトレードオフが存在する．加えて，メモリ制約の厳しいデバイスでは，展開鍵や中間値を保持する RAM 量，および spill/reload を含むデータメモリアクセスも重要となる．以上より，短い IoT ペイロードを効率的に処理するためには，HW/SW 双方の実装効率に加えて，低サイクル数，小コードサイズ，on-the-fly 鍵生成，低 RAM 使用量を同時に満たす 1 ブロック最適化設計が求められる．

1.3 本研究の貢献

本研究では，HW と SW の実装特性を設計段階から同時に考慮した 128-bit 軽量ブロック暗号を提案する．主な貢献を以下に示す．

- **HW/SW 同時最適化に基づく暗号設計：**HW 効率に優れる Midori の構成要素を基盤とし，Cortex-M3/M4 の命令セットとレジスタ制約を暗号設計に反映した．状態配置，Pack/Unpack，ShuffleCell，MixColumn および鍵更新を HW/SW 双方の観点から設計することで，on-the-fly 鍵更新を含む暗号化処理を 13 本の汎用レジスタ内で完結させ，入出力を除くデータメモリアクセスを不要とした．
- **HW/SW 双方における高い実装性能：**15 nm ASIC 評価では，低エネルギー HW 暗号の Midori-128 および低遅延 HW 暗号の QARMAv2-128 を含む比較対象に対し，回路規模，遅延，消費電力のすべてを削減した．また Cortex-M3/M4 では，1 ラウンドループ実装のまま Speck のアンロール実装と 4%以内の暗号化サイクル数を達成しつつ，コードサイズを約 90%削減し，Data RAM 0 byte を実現した．さらに AES-128 と比較しても HW/SW 双方で実装コストを大幅に削減した．

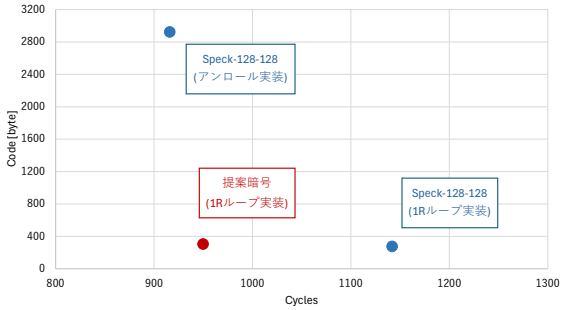


図 1 Cortex-M3 における Speck-128-128 の暗号化サイクル数とコードサイズのトレードオフおよび提案暗号のコードサイズとサイクル数比較．

一般に，組込み向け実装では，ループアンロールによる高速化とコードサイズの間にトレードオフが存在する．図 1 に，Cortex-M3 における暗号化サイクル数とコードサイズの関係を示す．Speck-128-128 では，1 ラウンドループ実装からアンロール実装への変更により，1142 cycles から 916 cycles へ高速化する一方，コードサイズは 276 byte から 2924 byte へ増加する．これに対し提案暗号は，1 ラウンドループ実装のまま 306 byte のコードサイズで 950 cycles を達成する．すなわち，Speck-128-128 のアンロール実装に対してコードサイズを 89.5%削減しながら，サイクル数の増加を 3.7%に抑えており，低コードサイズと低遅延を両立している．

2. 提案暗号の仕様

本章では，提案暗号の仕様について詳細を述べる．提案暗号は，128 ビットの平文 P およびマスター鍵 K を用いて 128 ビットの暗号文 C を出力するブロック暗号である．Substitution-Permutation Network（SPN）構造に基づき，HW および SW の双方における効率的な実装を対象としている．内部状態は，各 4 ビットの 32 個のセルから構成さ

© 2026 Information Processing Society of Japan

Algorithm 1 Encryption Overview.

```
1: procedure ENCRYPT(P, K)
2:     S ← Pack(P)
3:     K^(0) ← Pack(K)
4:     S ← KeyAdd(S, K^(0))
5:     for r = 1, ..., 19 do
6:         (S, K^(r)) ← ROUND(S, K^(r−1), r)
7:     end for
8:     S ← LASTROUND(S, K^(r))
9:     C ← Unpack(S)
10:    return C
11: end procedure
```

Algorithm 2
Round($r = 1, \ldots, 19$) & LastRound ($r = 20$)

```
1: procedure ROUND(S, K^(r−1), r)
2:     S ← SubCell(S)
3:     S ← ShuffleCell(S)
4:     S ← MixColumn(S)
5:     K^(r) ← KeyUpdate(K^(r−1), r)
6:     S ← KeyAdd(S, K^(r))
7:     return (S, K^(r))
8: end procedure
9:
10: procedure LASTROUND(S, K^(r))
11:    S ← SubCell(S)
12:    K^(20) ← KeyUpdate(K^(r), 20)
13:    S ← KeyAdd(S, K^(20))
14:    return S
15: end procedure
```

れ，フルラウンドは 20 ラウンドである．

2.1 暗号化の概要

提案暗号の構成を Algorithm 1 および 2 に，暗号化の全体像を図 A·1 に示す．Pack 後，初期 KeyAdd に続いて 19 回の通常ラウンドと 1 回の最終ラウンドを適用する．通常ラウンドは SubCell，ShuffleCell，MixColumn，KeyUpdate，KeyAdd から構成され，最終ラウンドでは ShuffleCell と MixColumn を省略する．

Pack 後の内部ステートを式 1 のように 4×8 セル配列として表す．ここで，各 s_i は 4 ビットのセルであり，左端のビットを MSB とする．

$$S = \begin{pmatrix} s_0 & s_4 & s_8 & s_{12} & s_{16} & s_{20} & s_{24} & s_{28} \\ s_1 & s_5 & s_9 & s_{13} & s_{17} & s_{21} & s_{25} & s_{29} \\ s_2 & s_6 & s_{10} & s_{14} & s_{18} & s_{22} & s_{26} & s_{30} \\ s_3 & s_7 & s_{11} & s_{15} & s_{19} & s_{23} & s_{27} & s_{31} \end{pmatrix} \tag{1}$$

2.2 Pack / Unpack

SW における bitsliced 実装 [6], [7], [8] を効率化するため，暗号化処理の入出力に bit permutation Pack/Unpack を適用する．128 ビット入力を $P = (p_0, p_1, \ldots, p_{127})$ とし，

表 1 Pack の bit 置換対応表

状態	bit 配置
P	$p_0, p_1, \ldots, p_{127}$
Pack(P)	$(p_0, p_{32}, p_{64}, p_{96},\ p_8, p_{40}, p_{72}, p_{104}, \ldots,$ $p_1, p_{33}, p_{65}, p_{97}, \ldots, p_{31}, p_{63}, p_{95}, p_{127})$

p_0 を MSB，p_{127} を LSB とする．Pack では，内部状態の行 r，列 c のセル s_{4c+r} の第 b ビットを

$$s_{4c+r}^{(b)} = p_{32b+8r+c}, \qquad 0 \le r, b < 4, \quad 0 \le c < 8$$

と定義する．Unpack は Pack の逆関数とし，具体的な bit 対応を表 1 に示す．

2.3 KeyUpdate

鍵スケジュール関数 KeyUpdate は，マスター鍵 K から各ラウンド鍵 $K^{(r)}$ を生成する．K には平文と同様に Pack を適用し，初期 KeyAdd に用いる鍵を $K^{(0)}$ とする．

ラウンド定数は各 4 ビット鍵セルの LSB に加算される．初期値 $RC^{(0)}$ と，その LSB を行優先順に並べた 32 ビット列 $\rho^{(0)}$ をそれぞれ

$$RC^{(0)} = \begin{pmatrix} 0 & 0 & 1 & 0 & 0 & 0 & 0 & 1 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 0 \\ 0 & 1 & 0 & 1 & 0 & 1 & 0 & 0 \\ 1 & 1 & 1 & 1 & 1 & 1 & 1 & 1 \end{pmatrix}$$

$$\rho^{(0)} = 00100001111111100101010011111111$$

と定義する．第 r ラウンドでは，

$$\rho^{(r)} = \mathrm{ROL}_{32}\left(\rho^{(0)}, r\right), \qquad 1 \le r \le 20,$$

を各鍵セルの LSB に配置した 128 ビット定数 $RC^{(r)}$ を用いて，

$$K^{(r)} = K^{(r-1)} \oplus RC^{(r)}$$

と更新する．ここで，$\mathrm{ROL}_{32}(x, l)$ は 32 ビットレジスタ x を l ビット左 rotate する処理を表す．

2.4 ラウンド関数

ラウンド関数は, SubCell, ShuffleCell, MixColumn, KeyAdd の 4 つのサブ関数から構成される．

SubCell.

SubCell は 4 ビットの入力からテーブル置換によって 4 ビット出力を得る S-box（非線形置換）である．提案暗号の S-box には，Midori の Sb0 を使用する．入出力の対応関係を表 2 に示す．

表 2 4 ビット S-box (16 進数表記)

x	0	1	2	3	4	5	6	7	8	9	a	b	c	d	e	f
SubCell(x)	c	a	d	3	e	b	f	7	8	9	1	5	0	2	4	6

© 2026 Information Processing Society of Japan

ShuffleCell.

ShuffleCell はセルの線形置換である．式 1 に示すステートを，表 3 に示す規則に従ってセルの配置を入れ替える．

表 3 ShuffleCell のセル置換 π．ここで $t_i = s_{\pi[i]}$ である．

i	0	1	2	3	4	5	6	7
$\pi[i]$	2	3	9	23	6	7	13	27
i	8	9	10	11	12	13	14	15
$\pi[i]$	10	11	17	31	14	20	21	0
i	16	17	18	19	20	21	22	23
$\pi[i]$	18	24	25	4	22	28	29	8
i	24	25	26	27	28	29	30	31
$\pi[i]$	26	1	15	12	30	5	19	16

MixColumn.

MixColumn は，ステートの各列に対して独立に適用する線形変換である．提案暗号では，Midori で用いられている Binary Matrix を使用する．式 1 に示すステートの各列に対して，以下の行列 M を乗算することでセルを混合する．

$0 \le c < 8$ に対して，MixColumn の変換を

$$\begin{pmatrix} s_{4c} \\ s_{4c+1} \\ s_{4c+2} \\ s_{4c+3} \end{pmatrix} \leftarrow M \begin{pmatrix} s_{4c} \\ s_{4c+1} \\ s_{4c+2} \\ s_{4c+3} \end{pmatrix}$$

として適用する．ここで，M は次の二値行列である．

$$M = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}.$$

行列演算における加算は 4 ビットセル単位の XOR である．

KeyAdd.

KeyAdd は，各ラウンドの最後にステートとラウンド鍵を XOR する．

パックされたセル順序における現在のラウンド鍵を

$$K^{(r)} = (K_0^{(r)}, K_1^{(r)}, \ldots, K_{31}^{(r)})$$

とすると，

$$s_i \leftarrow s_i \oplus K_i^{(r)}, \qquad 0 \le i < 32$$

として適用される．

3. 設計の過程

本章では，HW および SW の双方で高効率に実装可能な 128-bit ブロック暗号の設計過程を示す．本研究では，一方の環境に最適化した暗号構造を他方へ移植するのではなく，HW と SW で互いに不得手な演算を補完し，設計段階から双方の実装コストを考慮する HW/SW 同時最適化を基本方針とする．

具体的には，HW で低遅延かつ低消費電力な実装を指向して設計された Midori の S-box や Binary Matrix などを基礎部品として用い，HW 効率を維持する．その上で，4-bit セル表現を SW 上で扱う際に ボトルネックとなる状態配置，Pack/Unpack，ShuffleCell および MixColumn を，Cortex-M3/M4 の 32 ビットレジスタ, rotate, barrel shifter を活用できるよう再構成する．

以下では，まず対象環境から設計要件とレジスタ制約を導出し，その制約を引き継ぎながら各コンポーネントを順に決定する．

3.1 設計目標と設計要件

3.1.1 設計目標

本研究では，センサ情報などの短いペイロードを扱う Massive IoT デバイスを対象とし，ASIC や FPGA 等の専用 HW と Cortex-M3/M4 上の SW の双方で，1 ブロック暗号化を低遅延かつ低消費電力に実行可能な 128 ビットブロック暗号を設計する．第 1 章で述べた利用環境を踏まえ，以下を設計要件とする．

3.1.2 設計要件

要件 1 HW/SW 両環境で低消費電力に実装可能.

要件 2 1 ブロック暗号化の最適化.

要件 3 暗号化実行中のデータメモリへのアクセスなし.

要件 4 鍵スケジュールを on-the-fly で実行.

要件 5 コードサイズ削減のため, 1 ラウンドループ実装.

3.1.3 設計要件の妥当性

要件 1 および 2 に対して，HW 側では回路規模，critical path および消費電力を，SW 側では実行 cycle 数，命令数，レジスタ使用量およびデータメモリアクセスを主要な実装コストとして考慮する．

特に，固定 bit permutation は，HW では配線の接続関係を変更するだけで低コストに実現できる一方で，SW では，任意の bit permutation を実現するために mask，shift，rotate 等の複数命令を必要とする場合があり，命令数やレジスタ使用量の制約から実現可能な置換が強く制限される．そこで本研究では，Cortex-M3/M4 上で少数命令により実現可能な bit permutation の空間から候補を選択し，その中から十分な拡散性能を持つ構成を探索する．

また，短いペイロードでは鍵スケジュール等のコストを複数ブロックに分散できないため，要件 3–5 によって，on-the-fly 鍵更新を含む 1 ブロック暗号化全体の cycle 数，RAM 使用量およびコードサイズを抑える．

3.2 レジスタ制約と設計空間

要件 3–5 を満たすため，Cortex-M3/M4 で利用可能な 13 本の 32 ビット汎用レジスタ内に暗号化中の全データを保

© 2026 Information Processing Society of Japan

	row0			row1			row2			row3		
R[0]	$s_0^{(0)}$	$\cdots$	$s_{28}^{(0)}$	$s_1^{(0)}$	$\cdots$	$s_{29}^{(0)}$	$s_2^{(0)}$	$\cdots$	$s_{30}^{(0)}$	$s_3^{(0)}$	$\cdots$	$s_{31}^{(0)}$
R[1]	$s_0^{(1)}$	$\cdots$	$s_{28}^{(1)}$	$s_1^{(1)}$	$\cdots$	$s_{29}^{(1)}$	$s_2^{(1)}$	$\cdots$	$s_{30}^{(1)}$	$s_3^{(1)}$	$\cdots$	$s_{31}^{(1)}$
R[2]	$s_0^{(2)}$	$\cdots$	$s_{28}^{(2)}$	$s_1^{(2)}$	$\cdots$	$s_{29}^{(2)}$	$s_2^{(2)}$	$\cdots$	$s_{30}^{(2)}$	$s_3^{(2)}$	$\cdots$	$s_{31}^{(2)}$
R[3]	$s_0^{(3)}$	$\cdots$	$s_{28}^{(3)}$	$s_1^{(3)}$	$\cdots$	$s_{29}^{(3)}$	$s_2^{(3)}$	$\cdots$	$s_{30}^{(3)}$	$s_3^{(3)}$	$\cdots$	$s_{31}^{(3)}$

図 2 各レジスタへの packing 表現．ステート s_i の b-bit 目を $s_i^{(b)}$ と表記する．

持することを基本制約とする．128 ビットの内部状態とラウンド鍵にそれぞれ 4 本，ラウンド定数とループカウンタにそれぞれ 1 本を割り当てるため，10 本の用途が固定され，ラウンド関数内で自由に利用できるのは 3 本となる．したがって，ShuffleCell および SubCell は，状態 4 本を保持したまま最大 3 本の追加レジスタで実装可能でなければならない．

この制約の下で，各コンポーネントを以下の方針で決定する．

- MixColumn：Midori の Binary Matrix を基に，Cortex-M3/M4 で効率的な状態配置を決定する．
- Pack/Unpack：決定した状態配置を入出力 bit permutation へ組み込む．
- KeyUpdate：鍵レジスタ上で実行可能な on-the-fly 鍵更新を構成する．
- ShuffleCell：少数命令で実装可能かつ高い拡散性能を持つ置換を探索する．
- SubCell：HW 性能に優れる候補から，残されたレジスタ数で実装可能な S-box を選定する．

最終的なレジスタ割当てを表 4 に示す．

3.3 各コンポーネントの選定

3.3.1 MixColumn と内部状態配置

要件 1 および要件 2 を満たすため，線形層には Midori で用いられる Binary Matrix を採用する．bitsliced 状態において，同一列の 4 セルを 32 ビットレジスタ内の 8 ビット間隔に配置することで，各状態レジスタ x に対する MixColumn は

$$x' = \mathrm{ROL}_{32}(x, 8) \oplus \mathrm{ROL}_{32}(x, 16) \oplus \mathrm{ROL}_{32}(x, 24)$$

として実現できる．Cortex-M3/M4 では rotate をシフタオペランドとして利用できるため，4 本の状態レジスタに対する MixColumn を計 12 命令で実装できる．

この実装に合わせた状態配置を図 2 に，MixColumn の実装詳細を Appendix A.2 に示す．

状態行 r，列 c のセル s_{4c+r} を各 bitslice レジスタの

$$j = 8r + c$$

番目のビットに配置する．これにより，同一列の 4 セルが 8 ビット間隔に配置される．

3.3.2 入出力 Pack/Unpack

要件 1 および要件 2 を満たすため，前節で決定した内部状態配置に対応する入出力形式を決定する．一般的な bitsliced 実装 [8], [9] では，通常表現と bitsliced 表現の間の変換が必要であり，短い入力ではそのコストが無視できない．

そこで本暗号では，前節で決定した内部状態配置を暗号仕様上の Pack/Unpack として定義する．128 ビット入力を 4 本の 32-bit レジスタへロードした時点で bitsliced 状態となり，暗号化後も追加変換なしにそのままストアできる．これにより，SW では明示的な bitslice 変換を不要とし，1 ブロック暗号化の cycle 数を削減する．一方，HW では Pack/Unpack は配線のみで実現でき，追加の論理演算は不要である．

3.3.3 KeyUpdate

要件 4 を満たすため，ラウンド鍵は事前展開せず，4 本の鍵レジスタ上で on-the-fly に更新する．各セルの最下位ビットは 1 本の 32 ビットレジスタ $R[3]$ に格納されているため，32-bit のラウンド定数を $R[3]$ へ XOR することで，全 32 セルの最下位ビットにラウンド定数を一括して XOR できる．各ラウンドでは，ラウンド定数レジスタを 1 ビット左 rotate した後，$R[3]$ へ XOR することで鍵を更新する．この処理は rotate と XOR の 2 命令で実現でき，on-the-fly 実装においてもラウンド関数と比較して十分にコストを抑えられている．

3.3.4 ShuffleCell の設計空間と探索

ShuffleCell では，SW 実装コストを抑えつつ，十分な拡散性能を持つセル置換を探索する．拡散性能の目標として，Midori-128 と同様に，13 ラウンドで最小 Active S-box 数が 64 個以上となることを設定する．本暗号で用いる Sb0 の最大差分確率は 2^{-2} であるため，64 個の Active S-box は 2^{-128} の差分確率に対応する．

32 ビットレジスタ内のビット配置を変更する代表的な操作として，rotate と SWAPMOVE を考える．rotate はレジスタ全体の bit を循環移動する操作であるのに対し，SWAPMOVE は，距離 d だけ離れたビットペアのうち，マスク m で指定されたビットペアを同時に交換する操作であり，本稿では SWAPMOVE(d, m) と表す．詳細な演算方法は付録 A.3 に示す．SWAPMOVE ではマスク m を保持するレジスタが 1 本必要となるため，前節までのレジスタ制約の下では，ShuffleCell 内で使用する SWAPMOVE を 1

© 2026 Information Processing Society of Japan

表 4　13 本のレジスタに格納される情報

レジスタ	格納されている情報	補足
R[0]–R[3]	32bit ステート ×4 (=128 bit)	全てのセルの n-bit 目を R[n] に格納
R[4]–R[7]	32bit ラウンド鍵 ×4 (=128 bit)	on-the-fly で上書きして更新
R[8]	ループカウンタ	1 ラウンドループ実装のために必要
R[9]	ラウンド定数	KeyUpdate (on-the-fly) で使用
R[10]	SWAPMOVE 用 MASK 値	ShuffleCell 専用 MASK として使用
R[11]-R[12]	TMP0, TMP1	ラウンド関数および KeyUpdate における演算の一時変数として使用

表 5　最小 Active S-box 数の比較

	R1	R2	R3	R4	R5	R6	R7	R8	R9	R10	R11	R12	**R13**
提案暗号	1	4	6	12	17	24	33	38	40	46	52	60	**66**
Midori-128	–	–	–	16	23	30	35	38	41	50	57	62	**67**

表 6　各攻撃に対して安全となる最小ラウンド数

	差分／線形	不能差分	Boomerang
ラウンド数	13R	11R	9R

種類に固定する．

Cortex-M3/M4 では rotate を barrel shifter として他のデータ処理命令に組み込める一方，SWAPMOVE は 1 回当たり 5 命令を必要とする．したがって，SW 実装コストの観点では，4 本の状態レジスタを rotate のみで置換できる構成が理想的である．そこで，まず rotate のみ，あるいは一部のレジスタのみに任意の SWAPMOVE を適用する低コストな構成から探索する．

rotate のみで実現するシャッフルの探索．

探索の結果，4 本の状態レジスタのうち 1 本だけでも rotate のみで処理するようなシャッフルで，拡散性能の良いものは見つからなかった．32 ビットの rotate が存在すると，その 32 ビットだけで差分伝搬をコントロールすることが可能となり，最小 Active S-box 数が 1 ラウンド当たりおよそ 2 個ずつしか増加しない構成となることを確認した．このため，目標とする拡散性能を得るには，4 本すべての状態レジスタにおいてレジスタ内部のビットペアを交換する必要があることがわかる．つまり，全レジスタへの SWAPMOVE の適用が必要となる．

SWAPMOVE 命令を用いたシャッフルの探索．

そこで，4 本すべてに同一の SWAPMOVE を適用する条件の下で，交換距離 d およびマスク m の候補を探索した．交換可能なビットペアを多く含む候補から順に d を変化させて全探索し，各候補について SAT ソルバーによりラウンド数ごとの最小 Active S-box 数を評価した．その結果，SWAPMOVE$(5, \texttt{0x000007FF})$ を用いる構成において，Midori-128 と同程度の拡散性能が得られることを確認し，これを本暗号の ShuffleCell として採用する．

採用した ShuffleCell を用いた提案暗号と Midori-128 の最小 Active S-box 数を表 5 に示す．

この ShuffleCell では，現時点で利用可能な 3 本のレジスタのうち 1 本を SWAPMOVE 用マスクの保持に使用する．したがって，後段の SubCell で利用可能な一時レジスタは 2 本となる．

3.3.5　SubCell の設計空間と選定

要件 1 および要件 3 を満たすため，残された 2 本の一時レジスタという制約の下で SubCell を選定する．候補には，HW 実装性能を考慮して Orthros [10] の S-box と，Midori の Sb0，Sb1 を用いる．

各候補について，4 本の状態レジスタと最大 2 本の一時レジスタのみを用いた定数時間 bitsliced 実装の可否を評価した結果，この制約を満たしたのは Midori の Sb0 のみであった．したがって，本暗号の SubCell には Sb0 を採用する．

以上の過程を経て，提案暗号は HW/SW 同時最適化を目的として設計された．

4.　安全性評価

本章では，提案暗号に対する差分攻撃をベースとした攻撃への安全性を評価する．具体的には，差分/線形攻撃，不能差分攻撃，Boomerang 攻撃に対する 128bit 安全性を達成する最小ラウンドを評価する．各攻撃に対する結果について，表 6 に示す．以下では，それぞれの解析について詳細を説明する．

4.1　差分攻撃および線形攻撃

差分攻撃および線形攻撃に対する耐性を評価するため，各ラウンド数における最小 Active S-box 数を求めた．結果は表 5 に示す．本暗号の ShuffleCell の置換行列 P は $P^{-T} = P$ を満たし，MixColumn に用いる行列 M も対称かつ involutory であるため $M^{-T} = M$ を満たす．したがって，線形層 L について

$$L^{-T} = L$$

が成り立ち，差分伝搬と線形マスク伝搬における Active S-box の伝搬条件は一致する．このため，差分および線形 trail に対して同一の最小 Active S-box 数が得られる．

© 2026 Information Processing Society of Japan

表 7　15 nm ASIC 評価 (latency optimized)

アルゴリズム	ラウンド数	回路規模 [GE]	遅延 [ps]	処理電力 [mW]
提案暗号	20	31602	571	2.99
AES-128	10	142777	1026	31.14
Midori-128	20	35484	604	4.47
QARMAv2-128 ($r = 9$)	20	47261	623	4.96
QARMAv2-128 ($r = 11$)	24	56674	749	7.19

4.2　不能差分攻撃

不能差分攻撃 [11] は，入力差分 Δx から出力差分 Δy への伝搬確率が 0 となる差分対を利用する攻撃である．r ラウンドにおいて

$$\Pr[\Delta x \to \Delta y] = 0$$

を満たす差分対が存在するとき，これを r ラウンド不能差分特性と呼び，識別子や鍵回復攻撃に利用できる．
本暗号に対して探索した結果，10 ラウンドまでは不能差分特性が確認された一方，11 ラウンドでは該当する特性は得られなかった．したがって，本評価条件の下では，不能差分攻撃に利用可能な特性は 10 ラウンド以下に制限される．

4.3　Boomerang 攻撃

Boomerang 攻撃 [12], [13] では，前後の差分 trail の確率 p, q と中央部 E_m の接続確率 r から，Boomerang quartet の生成確率を $P_{\text{boom}} = p^2rq^2$ として評価する．
本評価では，E_m を 1 層の SubCell とし，BCT に基づく接続条件を考慮して，SAT ソルバーにより各 E_0/E_1 分割の最小 Active S-box 数を探索した．9 ラウンドでは最小 34 個となり，Midori Sb0 の最大差分確率 2^{-2} より $p^2q^2 \leq 2^{-136}$ を得る．$0 \leq r \leq 1$ であるため $p^2rq^2 \leq 2^{-136}$ も成り立ち，9 ラウンドで Boomerang trail に対する 128-bit の安全性基準を満たすことを確認した．解析の詳細は Appendix A.4 に示す．

5.　HW および SW における実装性能評価

本章では，提案暗号の HW および SW 実装性能を評価する．比較対象には，HW で低エネルギー実装を指向した Midori-128 ，128 ビットブロック・128 ビット鍵で最も低遅延 HW 実装の QARMAv2-128 ，SW 高速実装を指向した Speck-128-128 を用いる．また，広く利用される標準暗号である AES-128 を，HW/SW 双方に共通する基準として比較する．

5.1　HW 実装評価

評価環境．
15 nm 標準セルライブラリを用いて ASIC 実装評価を行い，回路規模（GE），クリティカルパス遅延（ps），消費電力（mW）を測定する．評価結果を表 7 に示す．
評価結果と考察．
提案暗号は 31,602 GE，571 ps，2.99 mW となり，表中のすべての比較対象に対して 回路規模，遅延，消費電力のいずれも小さい．特に，Midori-128-，QARMAv2-128 と比較しても 同等以上の性能を示している．また，標準暗号 AES-128 に対しては 3 指標すべてを大幅に削減している．以上より，提案暗号は HW 向けに設計された軽量暗号と比較しても競争力のある 低遅延・低消費電力 HW 実装を実現している．

5.2　SW 実装評価

評価環境．
SW では，Cortex-M3/M4 上で on-the-fly 鍵更新を含む 128 ビット 1 ブロックの性能を評価する．評価には STMicro NUCLEO F103RB (Cortex-M3) [*1]および NUCLEO-F411RE（Cortex-M4)[*2]を用い，FELICS [14], [15] の Scenario 0 に準じて 実行サイクル数，Data RAM 使用量およびコードサイズを測定する．
比較対象には，32 ビット MCU 上で高速な SW 実装を持つ Speck-128-128 の 1 ラウンドループ実装およびアンロール実装と，AES-128 の Fully-fixsliced 実装を用いる．Cortex-M3/M4 上の評価結果を表 8 に示す．
評価結果と考察．
提案暗号の 1 ラウンドループ実装は，暗号化で 950/946 cycles，復号で 968/965 cycles である．Speck-128-128 の 1 ラウンドループ実装より高速であり，暗号化では高速化のためにアンロールした Speck とも ほぼ同等のサイクル数で動作する．一方，提案暗号のコードサイズは 306/310 byte であり，Speck のアンロール実装の約 3 kB に対して大幅に小さい．復号では Speck のアンロール実装よりも少ないサイクル数を達成する．
さらに，内部状態，ラウンド鍵および一時値を 汎用レジスタ内に保持するため Data RAM 使用量は 0 byte であり，平文および鍵のロード後から暗号文のストアまで データメモリアクセスを必要としない．AES-128 の Fully-fixsliced 実装と比較すると，サイクル数，Data RAM 使用量およびコードサイズの すべてを大幅に削減している．

6.　まとめ

本研究では，短いペイロードを扱う Massive IoT を対象として，HW および SW の実装コストを設計段階から同時に考慮した 128-bit 軽量ブロック暗号を提案した．Midori の HW 効率に優れた構成要素を基盤としつつ, Cortex-M3/M4 の命令セットおよびレジスタ制約を反映することで，on-the-fly 鍵更新を含む暗号化処理を 13 本の汎用レジスタ内で完結させ，暗号化中のデータメモリアクセスを不要と

[*1] https://www.st.com/en/evaluation-tools/nucleo-f103rb.html
[*2] https://www.st.com/en/evaluation-tools/nucleo-f411re.html

© 2026 Information Processing Society of Japan

表 8　Cortex-M3/M4 における on-the-fly 実装評価結果．ただし，AES-128 は復号に関しては文献 [8] で未報告である．各値は Cortex-M3/Cortex-M4 の順に示す．

アルゴリズム	実装方法	Enc または Dec	Cycles	Data RAM [byte]	Code [byte]
提案暗号	1 ラウンドループ	Enc	950 / 946	0 / 0	306 / 310
		Dec	968 / 965	0 / 0	310 / 314
Speck-128-128	1 ラウンドループ	Enc	1142 / 1166	0 / 0	276 / 276
		Dec	1798 / 1826	0 / 0	384 / 392
	アンロール	Enc	916 / 938	0 / 0	2924 / 2968
		Dec	1168 / 1138	0 / 0	3200 / 3036
AES-128 (Fully-fixsliced)	1 ラウンドループ	Enc	5244 / 5308	352 / 352	3524 / 3524
		Dec	- / -	- / -	- / -
	アンロール	Enc	4806 / 4813	352 / 352	22112 / 22112
		Dec	- / -	- / -	- / -

した．

実装評価では，15 nm ASIC 上で Midori-128，QARMAv2-128 および AES-128 と比較して競争力のある低遅延・低消費電力性能を示した．また Cortex-M3/M4 上では，約 300 byte の小さなコードサイズと Data RAM 0 byte を維持しながら，Speck-128-128 の高速実装と同等水準の実行性能を達成し，AES-128 に対しては HW/SW 双方で実装コストを大幅に削減した．以上より，一方の実装環境に偏ることなく，単一の暗号プリミティブ上で HW/SW 双方に高い実装効率を実現できることを示した．

今後は，SWAPMOVE に限定した現在の ShuffleCell 探索空間を拡張し，Cortex-M3/M4 で利用可能な他の 32 ビットレジスタ操作も含めたより高速かつ高拡散な置換の探索を行う．

謝辞

本研究は，JST CRONOS JPMJCS25N1 および JSPS 科研費 JP24H00696 の支援を受けたものである．また，ハードウェア性能評価にご協力いただいた Lugano 大学 Subhadeep Banik 氏に深く感謝いたします．

参考文献

[1] LoRa Alliance: LoRaWAN, `https://lora-alliance.org/`. Accessed: 2026-08-21.

[2] Banik, S., Bogdanov, A., Isobe, T., Shibutani, K., Hiwatari, H., Akishita, T. and Regazzoni, F.: Midori: A Block Cipher for Low Energy, *ASIACRYPT 2015*, Lecture Notes in Computer Science, Vol. 9453, Springer, pp. 411–436 (2015).

[3] Avanzi, R., Banik, S., Dunkelman, O., Eichlseder, M., Ghosh, S., Nageler, M. and Regazzoni, F.: The QARMAv2 Family of Tweakable Block Ciphers, *IACR Transactions on Symmetric Cryptology*, Vol. 2023, No. 3, pp. 25–73 (2023).

[4] Beaulieu, R., Shors, D., Smith, J., Treatman-Clark, S., Weeks, B. and Wingers, L.: The SIMON and SPECK Lightweight Block Ciphers, *DAC 2015*, pp. 175:1–175:6 (2015).

[5] UnaBiz: Sigfox 0G Technology, `https://sigfox.com/`. Accessed: 2026-08-21.

[6] Biham, E.: A Fast New DES Implementation in Software, *Fast Software Encryption*, Lecture Notes in Computer Science, Vol. 1267, Springer, pp. 260–272 (1997).

[7] Käsper, E. and Schwabe, P.: Faster and Timing-Attack Resistant AES-GCM, *Cryptographic Hardware and Embedded Systems – CHES 2009*, Lecture Notes in Computer Science, Vol. 5747, Springer, pp. 1–17 (2009).

[8] Adomnicai, A. and Peyrin, T.: Fixslicing AES-like Ciphers New bitsliced AES speed records on ARM-Cortex M and RISC-V, *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, Vol. 2021, No. 1, pp. 402–425 (2021).

[9] Adomnicai, A., Najm, Z. and Peyrin, T.: Fixslicing: A New GIFT Representation Fast Constant-Time Implementations of GIFT and GIFT-COFB on ARM Cortex-M, *IACR Trans. Cryptogr. Hardw. Embed. Syst.*, Vol. 2020, No. 3, pp. 402–427 (2020).

[10] Banik, S., Isobe, T., Liu, F., Minematsu, K. and Sakamoto, K.: Orthros: A Low-Latency PRF, *IACR Trans. Symmetric Cryptol.*, Vol. 2021, No. 1, pp. 37–77 (2021).

[11] Biham, E., Biryukov, A. and Shamir, A.: Cryptanalysis of Skipjack Reduced to 31 Rounds Using Impossible Differentials, *EUROCRYPT*, Lecture Notes in Computer Science, Vol. 1592, Springer, pp. 12–23 (1999).

[12] Wagner, D. A.: The Boomerang Attack, *Fast Software Encryption, 6th International Workshop, FSE '99, Rome, Italy, March 24–26, 1999, Proceedings* (Knudsen, L. R., ed.), Lecture Notes in Computer Science, Vol. 1636, Springer, pp. 156–170 (1999).

[13] Cid, C., Huang, T., Peyrin, T., Sasaki, Y. and Song, L.: Boomerang Connectivity Table: A New Cryptanalysis Tool, *Advances in Cryptology – EUROCRYPT 2018* (Nielsen, J. B. and Rijmen, V., eds.), Lecture Notes in Computer Science, Vol. 10821, Springer, pp. 683–714 (2018).

[14] Dinu, D., Corre, Y. L., Khovratovich, D., Perrin, L., Großschädl, J. and Biryukov, A.: FELICS - Fair Evaluation of LIghtweight Cryptographic Systems, `https://www.cryptolux.org/index.php/FELICS` (2015).

[15] Dinu, D., Corre, Y. L., Khovratovich, D., Perrin, L., Großschädl, J. and Biryukov, A.: Triathlon of Lightweight Block Ciphers for the Internet of Things, *Journal of Cryptographic Engineering*, Vol. 9, No. 3, pp. 283–302 (2019).

© 2026 Information Processing Society of Japan

付　　録

A.1　暗号化の概要図

2 章で述べた暗号化の全体像を，図 A·1 に示す．

A.2　MixColumn の 32-bit レジスタ上での効率化

図 A·2 に，32-bit レジスタ上における MixColumn の実装方法を示す．本暗号では，状態を bitslice 形式で保持し，各 32-bit レジスタ内に状態行列の 4 行を 8 bit ずつ連続して配置する．例えば，先頭の 8 bit には $s_0, s_4, \ldots, s_{28}$，続く 8 bit には $s_1, s_5, \ldots, s_{29}$ が配置される．したがって，同一列に属する 4 セルは，レジスタ内で 8 bit 間隔の位置に対応する．

MixColumn では，Midori と同一の binary matrix

$$M = \begin{pmatrix} 0 & 1 & 1 & 1 \\ 1 & 0 & 1 & 1 \\ 1 & 1 & 0 & 1 \\ 1 & 1 & 1 & 0 \end{pmatrix}$$

を用いるため，各出力セルは同一列に属する自身以外の 3 セルの XOR として計算される．この性質と上述のレジスタ配置を組み合わせると，32-bit レジスタ x に対する MixColumn は

$$x' = \mathrm{ROL}_{32}(x, 8) \oplus \mathrm{ROL}_{32}(x, 16) \oplus \mathrm{ROL}_{32}(x, 24)$$

と表せる．

図 A·2 に示すように，8，16，24 bit のローテーションによって，レジスタ内の各 8-bit 領域がそれぞれ 1，2，3 行分だけ循環移動する．これらを XOR することで，各 bit 位置において元の状態行列の同一列に属する他の 3 行の値が重ね合わされ，MixColumn の出力を一括して得ることができる．

この方法では，セルごとの抽出や並べ替えを必要とせず，32-bit ワード全体に対するローテーションと XOR のみで 8 列分の MixColumn を並列に処理できる．特に Cortex-M3/M4 では，ローテーションを第 2 オペランドの barrel shifter として XOR 命令に組み込めるため，1 本の 32-bit レジスタに対する MixColumn を少数命令で実現できる．

A.3　SWAPMOVE による bit 置換

SWAPMOVE は，1 本のレジスタ内において，一定距離だけ離れた複数の bit 対を同時に交換する bit 置換である．32-bit 値 x に対して，交換する bit 間の距離を d，交換対象を指定する 32-bit マスクを m とすると，SWAPMOVE は

$$t = (x \oplus (x \gg d)) \wedge m,$$
$$x' = x \oplus t \oplus (t \ll d)$$

として計算できる．ここで，$\gg$ および $\ll$ はそれぞれ論理右シフトおよび論理左シフト，$\wedge$ は bitwise AND を表す．第 1 式では，距離 d だけ離れた bit 対の XOR を計算し，マスク m によって交換対象となる bit 対のみを抽出する．対象となる 2 bit が等しい場合には t の対応 bit は 0 となり，交換しても値は変化しない．一方，2 bit が異なる場合には対応 bit が 1 となり，第 2 式において下側および d bit 離れた上側の両 bit を反転することで，両者の値が交換される．したがって，複数の bit 対を個別に抽出して並べ替えることなく，同一距離 d の bit 交換をレジスタ全体に対して並列に実行できる．

例えば，マスク m の第 i bit が 1 である場合，x_i と x_{i+d} が交換対象となる．すなわち，SWAPMOVE は概念的には

$$(x_i, x_{i+d}) \longrightarrow (x_{i+d}, x_i) \qquad (m_i = 1)$$

を，マスクで指定されたすべての i について同時に適用する操作である．一方，$m_i = 0$ である bit 位置は変更されない．

本暗号の ShuffleCell では，32-bit レジスタ全体の rotate に加えて SWAPMOVE を用いることで，rotate のみでは実現できないレジスタ内部の bit 位置交換を行う．rotate がすべての bit 位置を一様に循環移動するのに対し，SWAPMOVE ではマスク m によって一部の bit 対のみを選択して交換できるため，より自由度の高いセル置換を構成できる．これにより，Cortex-M3/M4 上で少数の論理演算と shift のみを用いつつ，ShuffleCell に必要な拡散性能を実現する．

A.4　Boomerang 攻撃の詳細解析

Boomerang 攻撃 [12] では，暗号 E を前半 E_0 と後半 E_1 に分割し，それぞれにおける差分 trail を組み合わせて識別子を構成する．E_0 および E_1 における差分 trail の確率をそれぞれ p，q とすると，古典的な Boomerang quartet の生成確率は p^2q^2 で評価される．より厳密には，上下の差分 trail 間の依存性を考慮するため，中央部 E_m を導入する．E_m における接続確率を r とすると，Boomerang quartet の生成確率は

$$P_{\mathrm{boom}} = p^2 r q^2$$

で与えられる [13]．

本評価では，E_m を 1 層の SubCell とし，E_0 および E_1 の分割位置を変化させながら，SAT ソルバーを用いて Boomerang trail における最小 Active S-box 数を探索した．E_0 および E_1 では Midori Sb0 の DDT に基づく差分伝搬をモデル化し，E_m では Boomerang Connectivity Table (BCT) が非

© 2026 Information Processing Society of Japan

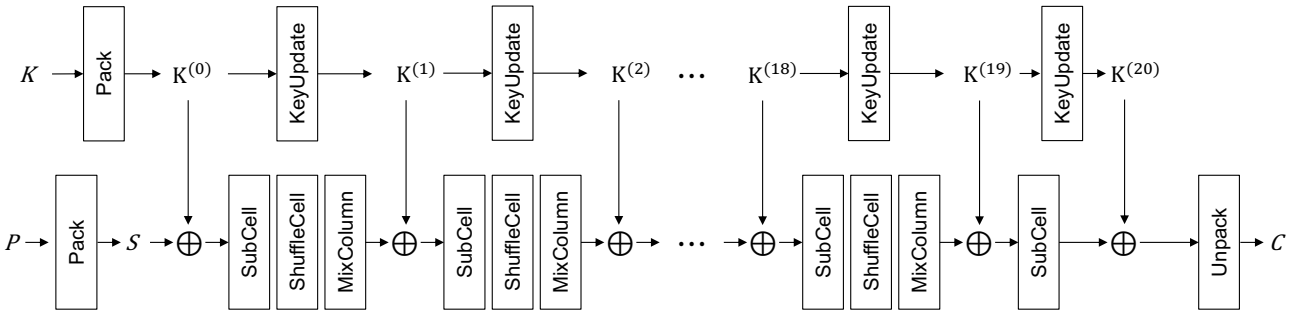


図 A·1　暗号化の概要図

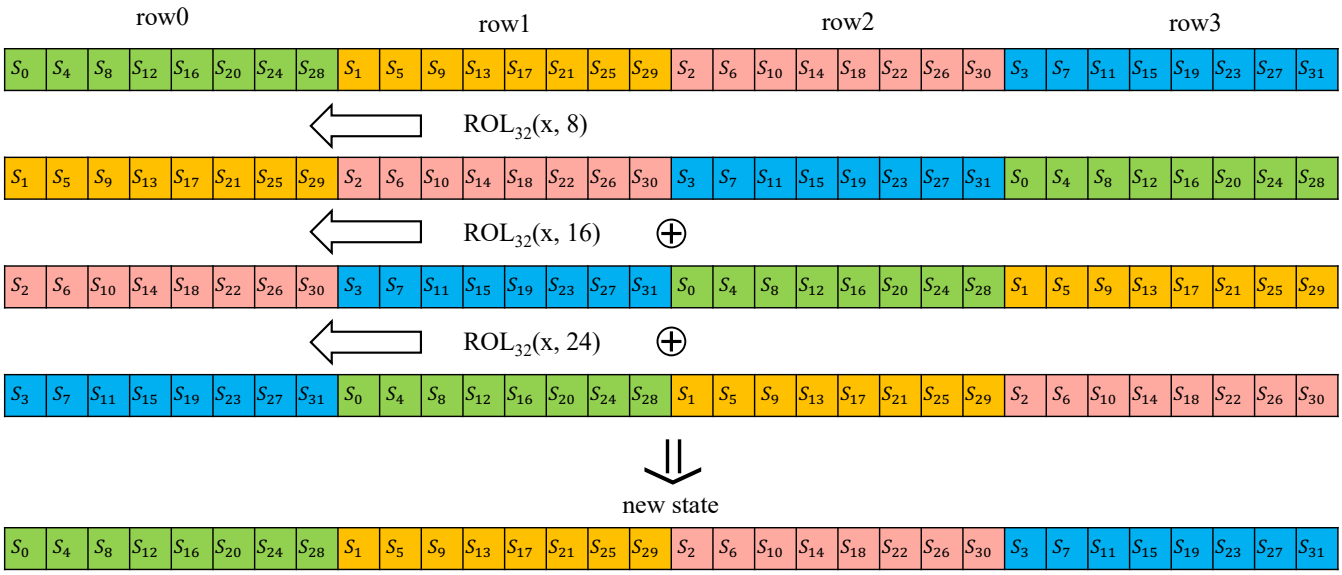


図 A·2　32 ビットレジスタ上における効率的な MixColumn 演算の概要図

零となる差分の組のみを許容した．これにより，E_m において接続確率 $r = 0$ となる上下の差分 trail を探索対象から除外した．

本暗号では 1 ラウンドに 1 層の SubCell を含む．また，E_0 および E_1 は中央の SubCell を共有するため，総ラウンド数 R は

$$R = r_{E_0} + r_{E_1} - 1$$

で与えられる．各 R について，この条件を満たすすべての (r_{E_0}, r_{E_1}) を探索した．

探索の結果，8，9，10 ラウンドにおける最小 Active S-box 数はそれぞれ 29，34，41 であった．9 ラウンドでは $(r_{E_0}, r_{E_1}) = (5, 5)$ で最小値 34 が得られ，10 ラウンドでは $(5, 6)$ または $(6, 5)$ で最小値 41 が得られた．

Midori Sb0 の最大差分確率は 2^{-2} である．E_0 および E_1 に含まれる Active S-box 数の総数を N とすると，p^2q^2 は

$$p^2q^2 \leq \left(2^{-2}\right)^{2N} = 2^{-4N}$$

で上界評価できる．したがって，8 ラウンドでは $N = 29$ より

$$p^2q^2 \leq 2^{-116}$$

であり，2^{-128} には到達しない．一方，9 ラウンドでは

表 A·1　Boomerang-Type 攻撃に対する最小 Active S-box 数．r_{E_0} および r_{E_1} は共有する中央 SubCell 層を含む．

$r_{E_0} \backslash r_{E_1}$	1	2	3	4	5	6	7	8	9	10
1	2	5	7	13	18	25	34	39	41	47
2	5	8	10	16	21	28	37	42	44	–
3	7	10	12	18	23	30	39	44	–	–
4	13	16	18	24	29	36	45	–	–	–
5	18	21	23	29	34	41	–	–	–	–
6	25	28	30	36	41	–	–	–	–	–
7	34	37	39	45	–	–	–	–	–	–
8	39	42	44	–	–	–	–	–	–	–
9	41	44	–	–	–	–	–	–	–	–
10	47	–	–	–	–	–	–	–	–	–

$N = 34$ より

$$p^2q^2 \leq 2^{-4\times 34} = 2^{-136}$$

となり，128-bit ブロックに対する 2^{-128} の基準を下回る．さらに，$0 \leq r \leq 1$ であることから

$$p^2rq^2 \leq p^2q^2 \leq 2^{-136}$$

が成り立つ．

© 2026 Information Processing Society of Japan

なお，本評価では BCT の非零条件を用いて E_m で接続可能な trail を制約しているが，BCT の値そのものを確率重みとして最適化してはいない．したがって，r を直接評価した p^2rq^2 の最適化ではなく，$r \leq 1$ とした保守的な上界評価である．それにもかかわらず，9 ラウンドにおいて $p^2q^2 \leq 2^{-136} < 2^{-128}$ が得られるため，9 ラウンド以降では Boomerang trail に対する 128-bit の安全性基準を満たすことを確認した．

© 2026 Information Processing Society of Japan